Chapter 1

Second-Order Systems

So far we've seen first-order differential equations and systems of first-order ODEs. In this chapter, we'll introduce second-order systems, which are particularly useful for modeling Newtonian motion.

1.1 Newtonian Motion

Newton's second law of motion is often written like this:

$$F = ma$$

where F is the net force acting on an object, m is the mass of the object, and a is the acceleration of the object.

This equation suggests that if you know m and a, you can compute the force. And that's true, but in most physical simulations it's the other way around: based on a physical model, you know F and m, and you compute a.

So if we know acceleration as a function of time, how do we find the position of the object, r? Well, we know that acceleration is the second derivative of position, so we can write the differential equation

$$\frac{d^2r}{dt^2} = a$$

where d^2r/dt^2 is the second time derivative of r.

Because this equation includes a second derivative, it's a second-order ODE. We can't solve the equation using `ode45` in this form, but by introducing a new variable, v, for velocity, we can rewrite it as a system of first-order ODEs:

$$\begin{aligned} \frac{dr}{dt} &= v \\ \frac{dv}{dt} &= a \end{aligned}$$

The first equation says that the first derivative of r is v; the second equation says that the first derivative of v is a.

Another way to present this model is to put it into linear *state space* form. The standard form for the state equation is

$$\dot{\mathbf{x}}(t) = A\mathbf{x}(t) + B\mathbf{u}(t) \tag{1.1}$$

where $\mathbf{x}(t)$ is the state vector of the system, $\mathbf{u}(t)$ is the input vector to the system, A is the state transition matrix and B is the input matrix. Again we've used the notation convention where scalar variables are expressed in regular font (e.g., t), vectors are in bold font (e.g., $\mathbf{x}$) and matrices are capitalized (e.g., A). This linear form is a special case of the more general non-linear state-space form in Equation 11.1

To get our system of equations into state space form we can define the state vector as a column vector with elements of position ($r(t)$) and velocity ($v(t)$).

$$\left\{ \begin{array}{c} \dot{r}(t) \\ \dot{v}(t) \end{array} \right\} = \left[\begin{array}{cc} 0 & 1 \\ 0 & 0 \end{array} \right] \left\{ \begin{array}{c} r(t) \\ v(t) \end{array} \right\} + \left[\begin{array}{c} 0 \\ 1 \end{array} \right] \{a\} \tag{1.2}$$

It turns out that any higher-order ordinary differential equation (ODE) can be transformed into a system of first-order ODEs. Numerical methods, such as `ode45` are often setup to operate on these systems of first-order ODEs. This example, of transforming a single second-order ODE into a system of two first-order ODEs is a specific example of a general approach.

1.2 Free Fall

As an example of Newtonian motion, let's go back to the question from Section 1.1:

> If you drop a penny from the top of the Empire State Building, how long does it take to reach the sidewalk, and how fast is it going when it gets there?

We'll start with no air resistance; then we'll add air resistance to the model and see what effect it has.

Near the surface of the earth, acceleration due to gravity is $-9.81\,\mathrm{m/s^2}$, where the minus sign indicates that gravity pulls down, so $a = -9.81\,\mathrm{m/s^2}$. If the object falls straight down, we can describe its position with a scalar value y, representing altitude.

Listing 1.1 contains a rate function we can use with `ode45` to solve this problem:

Listing 1.1: A rate function for the falling penny problem

```
function xx_dot = rate_func(t, xx)
    % Rate function using state space format
    A = [0, 1; 0, 0];
    B = [0; 1];
    uu = -9.81;

    xx_dot = A*xx+B*uu;
end
```

The rate function in Listing 1.1 takes scalar `t` and column vector `xx` as input variables, where the elements of `xx` are understood to be the position and velocity of the penny.

It returns a column vector that contains which are velocity and acceleration, respectively.

As always, we should test the rate function before we call `ode45`. Here's the top-level function we can use to test it:

```
% Initial conditions as state vector: position and velocity
xx_init = [381; 0];

% Test the rate function
rate_func(0, xx_init)
```

The initial condition of `xx` is the initial position, which is the height of the Empire State Building, about 381 m, and the initial velocity, which is 0 m/s.

The result from `rate_func` is

```
    0
   -9.8000
```

which is what we expect.

Now we can run `ode45` with this rate function:

```
tspan = [0, 10]
[tout, Xout] = ode45(@rate_func, tspan, xx_init);
```

As always, the first argument is the function handle, the second is the time span (10 seconds), and the third is the initial condition.

The result is a vector, `tout`, that contains the time values, and a matrix, `Xout`, that contains two columns, one for altitude and one for velocity.

We can extract the first column and plot it, like this:

```
Y = Xout(:, 1)
plot(tout, Y)
```

Here is the full listing...

```
% Simulate a penny falling from the Empire State Building.

% Initial conditions as state vector: position and velocity
xx_init = [381; 0];

% Test the rate function
rate_func(0, xx_init)

tspan = [0, 10];
[tout, Xout] = ode45(@rate_func, tspan, xx_init);

figure(1)
clf;
plot(tout,Xout(:,1))
xlabel('Time [s]')
ylabel('Position [m]')

function xx_dot = rate_func(t, xx)
    % Rate function using state space format
    A = [0, 1; 0, 0];
    B = [0; 1];
    uu = -9.81;

    xx_dot = A*xx+B*uu;
end
```

Figure 1.1 shows the result. Altitude drops slowly at first and picks up speed. Between 8 and 9 seconds, the altitude reaches 0, which means the penny hits the sidewalk. But ode45 doesn't know where the ground is, so the penny keeps going through 0 into negative altitude. We'll solve that problem in the next section.

1.3 ODE Events

Normally when you call ode45 you specify a start time and an end time. But sometimes you don't know ahead of time when the simulation should end. To solve this problem we can define an *event*, something of interest that happens during a simulation, like the penny reaching the ground.

Here are the steps:

1. First we define an *event function* that allows ode45 to figure out when an event occurs. Here's an event function for the penny example:

```
function [value, isterminal, direction] = event_func(t,xx)
    value = xx(1);
    isterminal = 1;
```

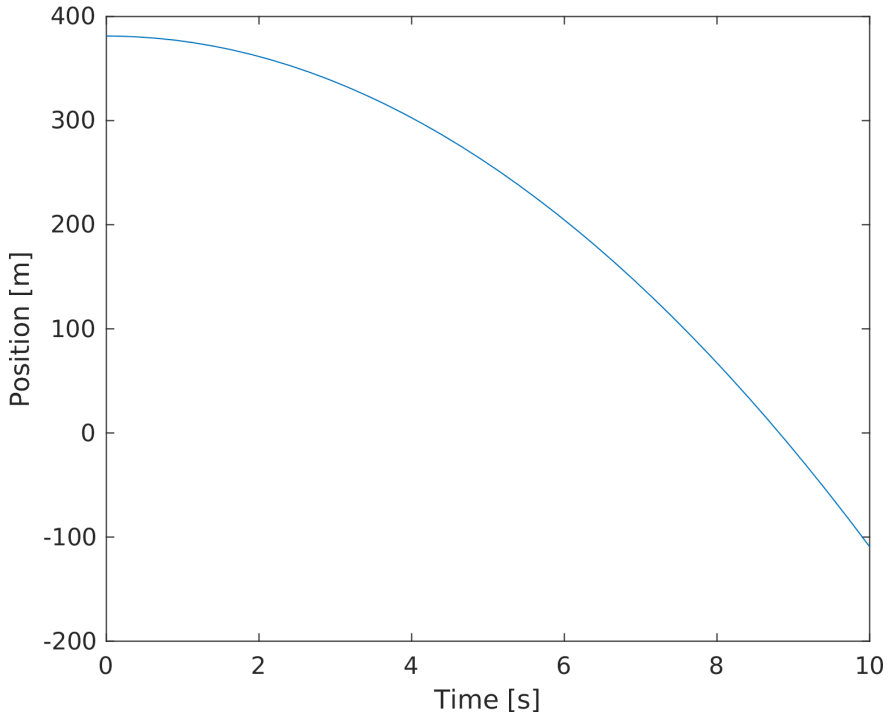


Figure 1.1: Altitude versus time for an object in free fall

```
    direction = -1;
end
```

The event function takes the same input variables as the rate function and returns three output variables: `value` is zero when an event can occur, `direction` can be 0 to allow all events, +1 for the event to occur only when the event function is increasing, and `isterminal` is 1 if the integration should terminate and 0 otherwise. More specifically, an event can occur when `value` passes through 0. If `direction` is positive, the event only occurs if `value` is increasing. If `direction` is negative, the event only occurs if `value` is decreasing. If `direction` is 0, the event always occurs. If `isterminal` is 1, the event causes the simulation to end; if it is 0, the simulation continues.

This event function uses the altitude of the penny as `value` so an event can occur when altitude passes through 0. Because `direction` is negative, an event occurs only when altitude is decreasing, and because `isterminal` is 1, the simulation ends if an event occurs.

2. Next, we use `odeset` to create an object called `options`:

```
options = odeset('Events', @event_func);
```

The name of the option is `Events` and the value is the handle of the event function.

3. Finally, we pass `options` as a fourth argument to `ode45`:

```
[tout, Xout] = ode45(@rate_func, tspan, xx_init, options);
```

When ode45 runs, it invokes event_func after each time step. If the event function indicates that a terminal event occurred, ode45 stops the simulation.

Let's look at the results from the penny example:

```
>> tout(end)
ans =
8.8134

>> Xout(end,:)
ans =
          0  -86.4594
```

The last value of tt is about 8.8, which is the number of seconds the penny takes to reach the sidewalk.

The last row of Xout indicates that the final altitude is 0, which is what we wanted, and the final velocity is about −86 m/s.

1.4 Air Resistance

To make this simulation more realistic, we can add air resistance. For large objects moving quickly through air, the force due to air resistance, *drag*, is proportional to velocity squared. For an object falling down, drag is directed up, so if velocity is negative, drag force is positive.

Here's how we can compute the force of drag as a function of velocity in one dimension:

$$f_d = -\text{sign}(v) b v^2$$

where v is velocity and b is a drag constant that depends on the density of air, the cross-sectional area of the object, and the shape of the object.

The sign or signum function returns the value 1 for positive values of v and −1 for negative values. So f_d is always in the opposite direction of v.

To convert from force to acceleration we have to know mass, but that's easy to find: the mass of a penny is about 2.5 g. It's not as easy to find the drag constant, but based on reports that the terminal velocity of a penny is about 18 m/s, I've estimated that it's about 75×10^{-6} kg/m.

Listing 1.2 defines a function that takes t and X as input variables and returns the total acceleration of the penny due to gravity and drag:

Listing 1.2: Calculating acceleration of a penny with drag

```
function accel = acceleration(t, xx)
    % Return total acceleration due to gravity and drag.

    % Drag constant in kg/m
```

```
    b = 75e-6;
    % Velocity from state, in m/s
    v = xx(2);
    % Drag force in N
    f_d = -sign(v) * b * v^2;
    % Mass in kg
    m = 2.5e-3;
    % Drag acceleration in m/s^2
    a_d = f_d / m;
    % Gravity
    a_g = -9.8;
    % Return total drag
    accel = a_g + a_d;
end
```

First, we compute force due to drag. Then we compute acceleration due to drag. Lastly, we compute total acceleration due to drag and gravity.

Be careful when you're working with forces and accelerations; make sure you only add forces to forces or accelerations to accelerations. In my code, I use comments to remind myself what units the values have. That helps me avoid errors like adding forces to accelerations.

To use this function, we make a small change in `rate_func`:

```
function xx_dot = rate_func(t, xx)
    % Rate function using state space format
    A = [0, 1; 0, 0];
    B = [0; 1];
    % Input is total acceleration - this line has changed!
    uu = acceleration(t, xx);

    xx_dot = A*xx+B*uu;
end
```

In the previous version, acceleration is always `-9.8`, the acceleration due to gravity. In this version, we call `acceleration` to compute the total acceleration due to gravity and drag.

Everything else is the same. Figure 1.2 shows the result.

Air resistance makes a big difference! Velocity increases until acceleration due to drag equals acceleration due to gravity; after that, velocity is constant and position decreases linearly (and much more slowly than it would in a vacuum).

With air resistance, the time until the penny hits the sidewalk is 22.4 s, substantially longer than before (8.8 s).

And the final velocity is 18.1 m/s, substantially slower than before (86 m/s).

Here is the full script...

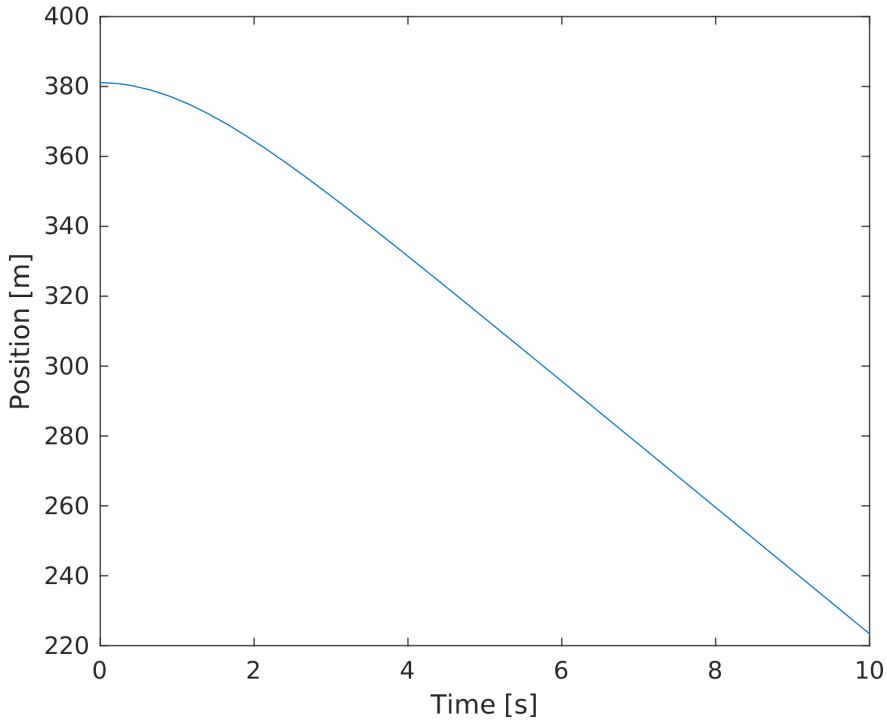


Figure 1.2: Altitude versus time for a penny in free fall with air resistance

```
% Simulate a penny falling from the Empire State Building.

% Initial conditions as state vector: position and velocity
xx_init = [381; 0];

% Test the rate function
rate_func(0, xx_init)

tspan = [0, 10];
options = odeset('Events', @event_func);
[tout, Xout] = ode45(@rate_func, tspan, xx_init, options);

figure(1)
clf;
plot(tout,Xout(:,1))
xlabel('Time [s]')
ylabel('Position [m]')

function xx_dot = rate_func(t, xx)
    % Rate function using state space format
    A = [0, 1; 0, 0];
    B = [0; 1];
    % Intput is total acceleration
```

```
    uu = acceleration(t, xx);

    xx_dot = A*xx+B*uu;
end

function accel = acceleration(t, xx)
    % Return total acceleration due to gravity and drag.

    % Drag constant in kg/m
    b = 75e-6;
    % Velocity from state, in m/s
    v = xx(2);
    % Drag force in N
    f_d = -sign(v) * b * v^2;
    % Mass in kg
    m = 2.5e-3;
    % Drag acceleration in m/s^2
    a_d = f_d / m;
    % Gravity
    a_g = -9.8;
    % Return total drag
    accel = a_g + a_d;
end

function [value, isterminal, direction] = event_func(t,xx)
    % Local event function for ode45 options
    value = xx(1);
    isterminal = 1;
    direction = -1;
end
```

1.5 Chapter Review

In this chapter, we used Newton's laws of motion to write a differential equation that describes the motion of a falling penny.

We rewrote that equation as a system of first-order differential equations, so we could use `ode45` to solve it. Then we ran simulations of a falling penny with and without air resistance, also known as *drag*.

We defined an *event* as something of interest that happens during a simulation, like a collision between moving objects, and we wrote an *event function*, which allows `ode45` to figure out when an event occurs.

In the next chapter, we extend Newtonian motion to two dimensions and model the flight of a baseball.

1.6 Exercises

Before you go on, you might want to work on the following exercise.

Exercise 1.1. In this exercise we'll model the descent of a skydiver, taking into account the change in drag when the parachute opens.

1. Modify the penny code from this chapter to simulate the descent of a 75 kg skydiver from an initial altitude of 4000 m. The drag constant for a skydiver without a parachute is about 0.2 kg/m. What would the velocity of the skydiver be on impact?
2. After opening their parachute, the velocity of the skydiver slows to about 5 m/s. Use your simulation to find the drag constant that yields a terminal velocity of 5 m/s.
3. Increase the mass of the skydiver, and confirm that terminal velocity increases. This phenomenon is the source of the intuition that heavy objects fall faster; in air, they do!
4. Now suppose the skydiver free falls until they get to an altitude of 1000 m before opening the parachute. How long would it take them to reach the ground?
5. What is the lowest altitude where the skydiver can open the parachute and still land at less than 6 m/s (assuming that the parachute opens and deploys instantly)?

Exercise 1.2. Here's a question from the website *Ask an Astronomer* (see `https://greenteapress.com/matlab/astro`):

> If the Earth suddenly stopped orbiting the Sun, I know eventually it would be pulled in by the Sun's gravity and hit it. How long would it take the Earth to hit the Sun? I imagine it would go slowly at first and then pick up speed.

Use `ode45` to answer this question. Here are some suggestions about how to proceed:

1. Look up the Law of Universal Gravitation and any constants you need. I suggest you work entirely in SI units: meters, kilograms, and newtons.
2. When the distance between the Earth and the Sun gets small, this system behaves badly, so you should use an event function to stop when the surface of the Earth reaches the surface of the Sun.
3. Express your answer in days, and plot the results as millions of kilometers versus days.